

Convolution-layer parameters optimization in Convolutional Neural Networks

Milad Kohzadi Chegeni^a, Abdolreza Rashno^a, Sadegh Fadaei^{b,*}

^a Department of Computer Engineering, Faculty of Engineering, Lorestan University, Khorramabad, Iran

^b Department of Electrical Engineering, Faculty of Engineering, Yasouj University, Yasouj, Iran

ARTICLE INFO

Article history:

Received 11 August 2022

Received in revised form 15 December 2022

Accepted 15 December 2022

Available online 17 December 2022

Keywords:

Convolutional Neural Networks

Parameters optimization

Particle swarm optimization

Convolution filter dimension

ABSTRACT

Convolutional Neural Networks (CNNs) are the most important deep learning algorithms to classify images based on their visual features. CNNs architectures are made of convolution, pooling and fully-connected (FC) layers and their corresponding parameters, which affect classification performance significantly. Convolution parameters optimization for CNNs, referred as CPOCNN, is proposed in this paper. To the best of our knowledge, this is the first optimization model which assigns adaptive upper-bounds of convolution parameters depends on data dimension in current layer and number of remained layers to reach the output layer. For this task, a comprehensive mathematical model is presented for constant structures of CNNs and it is proven that more optimization space is explored in comparison with all state-of-the-art methods. In optimization process, number of convolution filters and type of pooling filters are selected randomly; dimension of pooling filters, zero-padding and stride are considered constant. CPOCNN has been evaluated on 7 publicly available datasets and compared with 53 competitive CNN models with constant and optimized structures. Interested results showed that CPOCNN not only outperforms state-of-the-art CNN methods but also enriches weak CNN models by improving their accuracies more than 35%. Source Code of this paper is available online at Github.¹

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Convolutional neural networks (CNNs) are among the most successful methods for classification task. CNNs receive input data vectors (matrices), forward them for blocks of convolution and pooling layers, end with fully-connected (FC) layers to make desired outputs. Convolution layers extract local pattern of data and pooling is a down sampling operator. FC makes all-to-all connections between neurons. The important property of CNNs is local connections between neurons in blocks of convolution and pooling layers which enable them to have deeper structures with lower number of parameters in comparison with conventional artificial neural networks (ANNs) [1]. Main challenge of using CNNs in image classification is that: how CNNs structure can be designed to reach the best performance. Usually, experts in the field configure their networks by trial-and-test method which is a time-consuming task. Another way is to use successful networks such as AlexNet [2], VGGNet [3], ResNet [4], GoogleNet [5] and DenseNet [6] which have appropriate performance in standard benchmarks. Neural architecture search (NAS) is a method to

select appropriate architecture in search space by considering maximum number of layers and parameters of each layer in this space [7]. Generally, CNN optimization can be categorized to two sections architecture optimization and parameters optimization which are discussed in the next section.

1.1. Related works

In architecture optimization, order and type of layers, parameters of each layer including size, number and type of filters are optimized. A particle swarm optimization (PSO) method based on quantum binary encoding was proposed in [8] for CNN architecture optimization. In [9], a genetic algorithm architecture optimization method was presented based on variable-length gene encoding, novel representation method and a new fitness evaluation scheme. In [10], a PSO-based search method with novel encoding and velocity operators was introduced to find meaningful deep CNN structures for image classification. Number of parameters and test error of CNNs were formulated as a two-objective optimization problem in [11] which relies on hybrid binary encoding and penalty-based boundary intersection for structure optimization of CNNs. Classification accuracy was used as a reward in reinforcement learning for CNN structure optimization in [12]. In [13], IPPSO was presented by variable-length encoding of CNN structures and optimization by PSO. CNNs

* Corresponding author.

E-mail addresses: kohzadi.m@lu.ac.ir (M.K. Chegeni), rashno.a@lu.ac.ir (A. Rashno), s.fadaei@yu.ac.ir (S. Fadaei).

¹ <https://github.com/kohzadi/CHOCNN.git>

were encoded as fixed-length binary string and optimized by genetic algorithm [14].

In parameters optimization, convolution filter size, number of convolution filters, type and dimension of pooling filters, stride and zero-padding are optimized in CNNs with a constant structure. In [15], PSO parameter optimization method was introduced for pre-defined CNN architectures based on a confidence function and scalar acceleration coefficients. A parameter optimization was presented in [16] by enhanced metaheuristic algorithms. Automatic optimal parameter setting was proposed in [17] for CNNs based on PSO. In [18], evolutionary PSO algorithm was proposed for finding the best parameters of CNNs. In [19], a two-level PSO method was presented. In the first and second levels, CNN structure and layers parameters are optimized, respectively. Another two-level optimization algorithm was proposed in [20], in which, upper level minimizes the network complexity and the lower level optimizes the convolution block topologies. Finally, genetic algorithms was developed for parameters optimization of CNNs in [21].

CNN optimization is used in many applications. In [22], the topology of CNN was optimized based on the Grey Wolf Optimizer (GWO) and Artificial Bee Colony (ABC) algorithms named as hybrid GWO-ABC. Also, Flower Pollination Optimization Algorithm (FPOA) was used to optimize a CNN for feature extraction in [23]. A new method based on convolutional neural networks was proposed in [24] for fast static security assessment of power systems. In [25], a review on CNN optimization methods was provided. A new topology optimization approach for designing CNNs was introduced in [26]. Finally a novel combination of Bees Algorithm and Bayesian Optimization referred as BA-BO-CNN was proposed to optimize CNNs in [27].

1.2. Motivation and contributions of this research

All mentioned methods for CNN optimization need expert knowledge of domain. As a simple example, in optimization process, if dimension of kernels is selected with large values, data dimension is reduced significantly. Therefore, it should be controlled by user supervision. If user considers constant upper-bound M for dimension of all kernels, CNNs with parameters larger than M are not explored in optimization space. Furthermore, this bound should not be the same for all layers. In the first layers, larger bounds can be used, while in the last layers, this bound is more limited because of lower dimension of data. In fact, dimension of kernel in each layer depends on dimension of data in that layer, dimension of kernels, number and type of remained layers to reach to the last layer of the network. These challenges motivated us to do this research. In the other word, our motivation for this research stems from this question: How parameters of each CNN layer can be efficiently optimized without pre-defined user upper-bounds, with adaptive bounds depend on the position of that layer in the network? Contributions of this work are summarized as follows which present a parameters optimization model and provide a comprehensive answer to this question:

1. A constant structure for CNNs are considered. In this structure, CNNs include a series of subsequent blocks, each block contains L convolution layers and 1 pooling layer. As it will be explained later, CNN structures out of this structure also can be optimized.
2. Effect of convolution filter dimension in each layer is deeply investigated and a complete mathematical model is presented to compute maximum valid dimension for filter.

Table 1

Symbols for the proposed parameters bands determination.

Parameter	Symbol
Number of blocks	B
Minimum dimension of convolution kernel	$C_{min} \times C_{min}$
Dimension of convolution kernel in layer i	$C_i \times C_i$
Dimension of max pooling	P_0
Value of stride	S_0
Dimension of input image	$M_0 \times M_0$
Image dimension in layer i	$M_i \times M_i$
Maximum number of kernels	NK_{max}
Minimum number of kernels	NK_{min}
Number of kernels in layer i	NK_i

3. Proposed model uses two considerations for filter dimension bound computation in layer i : (1) How much data dimension has been reduced from input layer to i th layer, and (2) Larger filter dimensions are allowed until data can be passed from layer i to the output layer with the minimum filter dimension.
4. As a property of CPOCNN, it is proven that adaptive upper-bounds of filter dimensions in each CNN layer make more exploration of optimization space in comparison with constant upper-bounds used in existing optimization methods.
5. Number of convolution filters and type of pooling filters (max/ave) does not affect data reduction rate, therefore they are selected randomly in optimization process. Parameters including dimension of pooling filters, zero-padding and stride are not involved in optimization process.
6. Proposed model focuses on convolution parameter optimization (CPO), therefore it is referred as CPOCNN.
7. Results show that optimization of *only* convolution parameters improves CNN performance significantly specially in weak CNN structures in challenging datasets.
8. Primary insights for optimization of other CNN parameters and optimization of CNNs with different block structures are directed as future works.

The rest of this paper is organized as follows: CPOCNN model is described in Section 2. Experimental setup and results are presented in Section 3. Discussion and future works are discussed in Section 4. Finally, the paper is concluded in Section 5.

2. Proposed method

The main contribution of this research is parameters optimization. For this task PSO is reformulated as a single-objective optimization method to optimize parameters of a constant-structure CNN. Constant-structure refers to networks with constant order, number and type of layers, while kernel dimension, number and type of kernels in each layer are optimized in optimization process.

2.1. Basic symbols

In the first step, maximum degree of freedom is calculated for parameters of each layer. It is worth mentioning that since number and type of layers is constant during optimization process, it is very important to use appropriate parameters to preserve compressed data until the last layer. Otherwise, high kernel dimensions in convolution layers without padding for example, reduce input data dimension quickly and do not allow to be forwarded to the last layer. To handle this challenge a model is proposed for automatic determination of parameters upper-bounds in each layer. For this task, CNN parameters are defined with symbols as reported in Table 1.

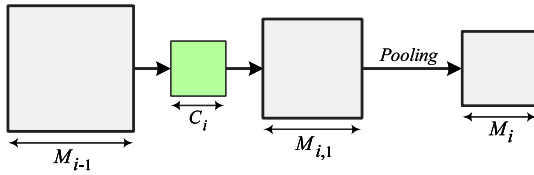


Fig. 1. One block of CNN containing one convolution layer and one pooling layer.

Conventional structures of CNNs are sequence of blocks, each block contains convolution layers ended by a pooling layer. Therefore, different cases of CNN structures are analyzed and a generalization scheme is presented. Note that zero-padding is not used in all models, dimension of max-pooling and stride are considered as constant values P_0 and S_0 in all layers, respectively.

2.2. One convolution layer and one pooling layer in each block

In the first case, each CNN block includes one convolution layer and one pooling layer as shown in Fig. 1. If input data M_{i-1} in i th block of CNN is passed from convolution layer with kernel size C_i , output data dimension $M_{i,1}$ is computed as:

$$M_{i,1} = M_{i-1} - C_i + 1 \quad (1)$$

For pooling layer with constant kernel size P_0 and constant stride S_0 input data $M_{i,1}$ is mapped to output M_i as follows:

$$M_i = \left\lfloor \frac{M_{i,1} - (P_0 - S_0)}{S_0} \right\rfloor = \left\lfloor \frac{M_{i,1} - P_0 + S_0}{S_0} \right\rfloor \quad (2)$$

where $\lfloor \cdot \rfloor$ is lower integer bound operation. If input data M_{i-1} is passed from convolution and pooling layers of a block, output is M_i and computed as follows:

$$M_i = \left\lfloor \frac{M_{i-1} - C_i + 1 - P_0 + S_0}{S_0} \right\rfloor \quad (3)$$

CNNs behave like data compression tools since they reduce data dimension in each layer. Reduction rate depends on CNN parameters such as stride and dimension of kernels. In existing optimization methods, parameters are selected randomly in a constant interval supervised by users. Therefore, they may reduce data dimension more than usual and leads to empty data in middle layer of CNN. In such cases, users reduce upper-bound of the interval manually. Proposed CPOCNN guarantees that computed upper-bound of parameters in each layer makes a non-empty data in the last layer which its dimension is greater than or equal to 1 (dimension 1 is considered as the worst case).

$$M_B \geq 1 \quad (4)$$

For example, suppose that CNN parameters are set to $B = 10$, $M_0 = 32$, $C_{min} = 3$, $C_{max} = 13$, $P_0 = 5$ and $S_0 = 3$. If this CNN structure starts to train with $C_i = 7$, based on Eq. (3), output dimension of each layer is as follows:

$$M_1 = \left\lfloor \frac{M_0 - C_1 + 1 - P_0 + S_0}{S_0} \right\rfloor = \left\lfloor \frac{32 - 7 + 1 - 5 + 3}{3} \right\rfloor = 8 \quad (5)$$

$$M_2 = \left\lfloor \frac{M_1 - C_2 + 1 - P_0 + S_0}{S_0} \right\rfloor = \left\lfloor \frac{8 - 7 + 1 - 5 + 3}{3} \right\rfloor = 0 \quad (6)$$

It means that data is empty in the second layer while 10 layers ($B = 10$) are configured for the training of this network. It is

needed that the dimension of data before fully-connected layers is more than 1. This example shows the importance of parameter configuration in the optimization of constant-structure CNNs.

To determine appropriate values of parameters in layer i , the worst case of parameters in layers $i+1, i+2, \dots, B$ are considered as:

$$\begin{aligned} \{C_{i+1}, C_{i+2}, \dots, C_B\} &= C_{min} \\ \{P_{i+1}, P_{i+2}, \dots, P_B\} &= P_0 \\ \{S_{i+1}, S_{i+2}, \dots, S_B\} &= S_0 \end{aligned} \quad (7)$$

The main reason for this consideration is that parameters of layer i is computed as upper-bounds and not always are selected with the maximum values. It is clear that if parameters of layer i , are randomly selected as maximum values (upper-bounds) in worst case, parameters of all remained layers are enforced to be configured with their lower-bounds. To determine data dimension in the last layer, Eq. (3) is used. Since this equation has operator $\lfloor \cdot \rfloor$, it is difficult to compute lower and upper bounds of parameters. To address this challenge, the following inequality is applied instead of $\lfloor \cdot \rfloor$.

$$\lfloor x \rfloor \geq x - 1 \quad (8)$$

Therefore, Eq. (3) is rewritten as:

$$\begin{aligned} M_i &= \left\lfloor \frac{M_{i-1} - C_i + 1 - P_0 + S_0}{S_0} \right\rfloor = \left\lfloor \frac{M_{i-1} - C_i - P_0 + 1}{S_0} + 1 \right\rfloor \\ \Rightarrow M_i &\geq \frac{M_{i-1} - C_i - P_0 + 1}{S_0} \end{aligned} \quad (9)$$

Blocks 1, 2, $\dots$, B are computed as follows:

$$M_1 \geq \frac{M_0 - C_1 - P_0 + 1}{S_0} \quad (10)$$

$$\begin{aligned} M_2 &\geq \frac{M_1 - C_2 - P_0 + 1}{S_0} \\ M_2 &\geq \frac{\frac{M_0 - C_1 - P_0 + 1}{S_0} + 1 - C_2 - P_0}{S_0} = \\ &= \frac{M_0 - (C_1 + S_0 C_2) - P_0(1 + S_0) + 1 + S_0}{S_0^2} = \\ &= \frac{M_0 - (C_1 + S_0 C_2) - (P_0 - 1)(1 + S_0)}{S_0^2} \\ \Rightarrow M_2 &\geq \frac{M_0 - (C_1 + S_0 C_2) - (P_0 - 1)(1 + S_0)}{S_0^2} \end{aligned} \quad (11)$$

$$\begin{aligned} M_3 &\geq \frac{M_2 - C_3 - P_0 + 1}{S_0} \\ M_3 &\geq \frac{\frac{M_0 - (C_1 + S_0 C_2) - (P_0 - 1)(1 + S_0)}{S_0^2} - C_3 - P_0 + 1}{S_0} \\ M_3 &\geq \frac{M_0 - (C_1 + S_0 C_2 + S_0^2 C_3) - (P_0 - 1)(1 + S_0 + S_0^2)}{S_0^3} \end{aligned} \quad (12)$$

Therefore, for i th layer, it can be formulated as:

$$M_i \geq \frac{M_0 - \sum_{m=1}^i S_0^{(m-1)} C_m - (P_0 - 1) \sum_{m=1}^i S_0^{(m-1)}}{S_0^i} \quad (13)$$

After simplification we have:

$$M_i \geq \frac{M_0 - \sum_{m=1}^i S_0^{(m-1)} (C_m + P_0 - 1)}{S_0^i} \quad (14)$$

For the last layer:

$$M_B \geq \frac{M_0 - \sum_{m=1}^B S_0^{(m-1)}(C_m + P_0 - 1)}{S_0^B} \quad (15)$$

To have non-empty data in the last layer, constraint in Eq. (4) should be satisfied:

$$\begin{aligned} M_B &\geq 1 \\ \Rightarrow \frac{M_0 - \sum_{m=1}^B S_0^{(m-1)}(C_m + P_0 - 1)}{S_0^B} &\geq 1 \\ \Rightarrow M_0 - \sum_{m=1}^B S_0^{(m-1)}(C_m + P_0 - 1) &\geq S_0^B \\ \Rightarrow \sum_{m=1}^B S_0^{(m-1)}(C_m + P_0 - 1) + S_0^B &\leq M_0 \end{aligned} \quad (16)$$

Providing parameters of all layers, Eq. (16) guarantees that data in the last layer is non-empty and optimization process is not stopped because of dimension errors. In the next step, it is investigated that how an interval is computed for dimension of i th convolution layer. For this purpose, Eq. (9) is extended as:

$$M_i \geq \frac{M_{i-1} - C_i - P_0 + 1}{S_0} \quad (17)$$

$$\begin{aligned} M_{i+1} &\geq \frac{M_i - C_{i+1} - P_0 + 1}{S_0} \\ \Rightarrow M_{i+1} &\geq \frac{\frac{M_{i-1} - C_i - P_0 + 1}{S_0} - C_{i+1} - P_0 + 1}{S_0} = \\ &\quad \frac{M_{i-1} - C_i - P_0 + 1 - S_0 C_{i+1} - S_0 P_0 + S_0}{S_0^2} \\ \Rightarrow M_{i+1} &\geq \frac{M_{i-1} - C_i - S_0 C_{i+1} - (P_0 - 1)(1 + S_0)}{S_0^2} \end{aligned} \quad (18)$$

$$\begin{aligned} M_{i+2} &\geq \frac{M_{i+1} - C_{i+2} - P_0 + 1}{S_0} \\ &\quad \frac{M_{i-1} - C_i - S_0 C_{i+1} - (P_0 - 1)(1 + S_0)}{S_0^2} - C_{i+2} - P_0 + 1 \\ M_{i+2} &\geq \frac{\frac{M_{i-1} - C_i - S_0 C_{i+1} - (P_0 - 1)(1 + S_0)}{S_0^2} - C_{i+2} - P_0 + 1}{S_0} = \\ &\quad \frac{M_{i-1} - C_i - S_0 C_{i+1} - (P_0 - 1)(1 + S_0) - S_0^2 C_{i+2} - S_0^2 P_0 + S_0^2}{S_0^3} \\ M_{i+2} &\geq \frac{M_{i-1} - C_i - (S_0 C_{i+1} + S_0^2 C_{i+2}) - (P_0 - 1)(1 + S_0 + S_0^2)}{S_0^3} \end{aligned} \quad (19)$$

Based on Eqs. (17), (18) and (19), in layer B:

$$M_B \geq \frac{M_{i-1} - C_i - \sum_{m=i+1}^B S_0^{(m-i)} C_m - (P_0 - 1) \sum_{m=i}^B S_0^{(m-i)}}{S_0^{(B-i+1)}} \quad (20)$$

Applying the most important constrain declared in Eq. (4):

$$\begin{aligned} M_B &\geq 1 \\ \Rightarrow \frac{M_{i-1} - C_i - \sum_{m=i+1}^B S_0^{(m-i)} C_m - (P_0 - 1) \sum_{m=i}^B S_0^{(m-i)}}{S_0^{(B-i+1)}} &\geq 1 \\ \Rightarrow M_{i-1} - C_i - \sum_{m=i+1}^B S_0^{(m-i)} C_m - (P_0 - 1) \sum_{m=i}^B S_0^{(m-i)} &\geq S_0^{(B-i+1)} \\ \Rightarrow C_i &\leq M_{i-1} - \sum_{m=i+1}^B S_0^{(m-i)} C_m - (P_0 - 1) \sum_{m=i}^B S_0^{(m-i)} - S_0^{(B-i+1)} \end{aligned} \quad (21)$$

Based on availability of M_{i-1} and considering the minimum values for C_m in the next block ($C_m = C_{min}$), intervals of C_i are computed as:

$$C_i \leq M_{i-1} - \sum_{m=i+1}^B S_0^{(m-i)} C_{min} - (P_0 - 1) \sum_{m=i}^B S_0^{(m-i)} - S_0^{(B-i+1)} \quad (22)$$

For easier representation of Eq. (22), right-hand side is substituted by Q_i :

$$Q_i = M_{i-1} - C_{min} \sum_{m=i+1}^B S_0^{(m-i)} - (P_0 - 1) \sum_{m=i}^B S_0^{(m-i)} - S_0^{(B-i+1)} \quad (23)$$

Therefore, the Eq. (22) is rewritten as:

$$C_i \leq Q_i \quad (24)$$

The main purpose is to determine acceptable interval for C_i :

$$C_i \in [C_{min}, Q_i] \quad (25)$$

Generally, Eq. (25) is used to compute intervals C_i . Please note that in each layer, values of parameters are selected randomly from their corresponding intervals in optimization process.

2.3. Two convolution layers and one pooling layer in each block

In this case, each CNN block contains two convolution layers and one pooling layer as shown in Fig. 2. Based on Eq. (2) for input data M_{i-1} in i th block, output of the first convolution layer is:

$$M_{i,1} = M_{i-1} - C_{i,1} + 1 \quad (26)$$

where $C_{i,1}$ and $M_{i,1}$ represent dimension of kernel in the first convolution layer and output, respectively. Output of the second convolution layer is computed as follows:

$$\begin{aligned} M_{i,2} &= M_{i,1} - C_{i,2} + 1 = M_{i-1} - C_{i,1} + 1 - C_{i,2} + 1 \\ &= M_{i-1} - (C_{i,1} + C_{i,2}) + 2 \end{aligned} \quad (27)$$

where $M_{i,2}$ and $C_{i,2}$ are respectively output and dimension of kernel in the second convolution layer. Output of the pooling layer; as output of the i th block; is computed as follows:

$$\begin{aligned} M_i &= \left\lfloor \frac{M_{i,2} - P_0 + S_0}{S_0} \right\rfloor \\ &= \left\lfloor \frac{M_{i-1} - (C_{i,1} + C_{i,2}) + 2 - P_0 + S_0}{S_0} \right\rfloor \\ &= \left\lfloor \frac{M_{i-1} - (C_{i,1} + C_{i,2}) + 2 - P_0}{S_0} + 1 \right\rfloor \end{aligned} \quad (28)$$

Based on Eq. (8):

$$M_i \geq \frac{M_{i-1} - (C_{i,1} + C_{i,2}) + 2 - P_0}{S_0} \quad (29)$$

In the next step, Eq. (29) is computed for $i+1$, $i+2$ and so on (Eq. (31) is given in Box 1):

$$\begin{aligned} M_{i+1} &\geq \frac{M_i - (C_{i+1,1} + C_{i+1,2}) + 2 - P_0}{S_0} \\ M_{i+1} &\geq \frac{\frac{M_{i-1} - (C_{i,1} + C_{i,2}) + 2 - P_0}{S_0} - (C_{i+1,1} + C_{i+1,2}) + 2 - P_0}{S_0} \\ M_{i+1} &\geq \frac{M_{i-1} - (C_{i,1} + C_{i,2}) - S_0(C_{i+1,1} + C_{i+1,2}) - (P_0 - 2)(1 + S_0)}{S_0^2} \end{aligned} \quad (30)$$

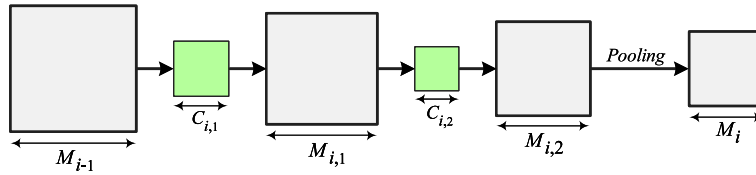


Fig. 2. One block of CNN with two convolution layers and one pooling layer.

$$\begin{aligned}
 M_{i+2} &\geq \frac{M_{i+1} - (C_{i+2,1} + C_{i+2,2}) + 2 - P_0}{S_0} \\
 M_{i+2} &\geq \frac{\frac{M_{i-1} - (C_{i,1} + C_{i,2}) - S_0(C_{i+1,1} + C_{i+1,2}) - (P_0 - 2)(1 + S_0)}{S_0^2} - (C_{i+2,1} + C_{i+2,2}) + 2 - P_0}{S_0} \\
 M_{i+2} &\geq \frac{M_{i-1} - (C_{i,1} + C_{i,2}) - S_0(C_{i+1,1} + C_{i+1,2}) - S_0^2(C_{i+2,1} + C_{i+2,2}) - (P_0 - 2)(1 + S_0 + S_0^2)}{S_0^3}
 \end{aligned} \quad (31)$$

Box 1.

Based on Eqs. (30) and (31) for B th block, it can be concluded that:

$$M_B \geq \frac{M_{i-1} - (C_{i,1} + C_{i,2}) - \sum_{m=i+1}^B S_0^{m-i}(C_{m,1} + C_{m,2}) - (P_0 - 2) \sum_{m=i}^B S_0^{m-i}}{S_0^{(B-i+1)}} \quad (32)$$

Applying primary constraint in Eq. (4) leads to:

$$\begin{aligned}
 M_B &\geq 1 \\
 \Rightarrow \frac{M_{i-1} - (C_{i,1} + C_{i,2}) - \sum_{m=i+1}^B S_0^{m-i}(C_{m,1} + C_{m,2}) - (P_0 - 2) \sum_{m=i}^B S_0^{m-i}}{S_0^{(B-i+1)}} &\geq 1 \\
 \Rightarrow M_{i-1} - (C_{i,1} + C_{i,2}) - \sum_{m=i+1}^B S_0^{m-i}(C_{m,1} + C_{m,2}) - (P_0 - 2) \sum_{m=i}^B S_0^{m-i} &\geq S_0^{(B-i+1)} \\
 \Rightarrow (C_{i,1} + C_{i,2}) &\leq M_{i-1} - \sum_{m=i+1}^B S_0^{m-i}(C_{m,1} + C_{m,2}) - (P_0 - 2) \sum_{m=i}^B S_0^{m-i} - S_0^{(B-i+1)}
 \end{aligned} \quad (33)$$

Based on Eq. (33), availability of M_{i-1} and considering minimum values for $C_{m,1}$ and $C_{m,2}$ in the next blocks, intervals for $C_{i,1}$ and $C_{i,2}$ can be computed as follows:

$$C_{i,1} + C_{i,2} \leq M_{i-1} - 2C_{min} \sum_{m=i+1}^B S_0^{m-i} - (P_0 - 2) \sum_{m=i}^B S_0^{m-i} - S_0^{(B-i+1)} \quad (34)$$

And,

$$Q_i = M_{i-1} - 2C_{min} \sum_{m=i+1}^B S_0^{m-i} - (P_0 - 2) \sum_{m=i}^B S_0^{m-i} - S_0^{(B-i+1)} \quad (35)$$

Therefore:

$$C_{i,1} + C_{i,2} \leq Q_i \quad (36)$$

There are two unknown variables $C_{i,1}$ and $C_{i,2}$ which should satisfy inequality (36). For this condition, first $C_{i,2} = C_{min}$ is considered to determine upper-bound of $C_{i,1}$ as follows:

$$C_{i,1} \leq Q_i - C_{min} \quad (37)$$

Therefore, acceptable interval for $C_{i,1}$ is:

$$C_{i,1} \in [C_{min}, Q_i - C_{min}] \quad (38)$$

After random selection of convolution kernel dimension in interval Eq. (38), $C_{i,2}$ is computed as:

$$C_{i,2} \in [C_{min}, Q_i - C_{i,1}] \quad (39)$$

2.4. Generalization to L convolution layers and one pooling layer in each block

Based on Eqs. (35) and (36), Q_i for 3 convolution layers and 1 pooling layer in each block can be computed as:

$$Q_i = M_{i-1} - 3C_{min} \sum_{m=i+1}^B S_0^{m-i} - (P_0 - 3) \sum_{m=i}^B S_0^{m-i} - S_0^{(B-i+1)} \quad (40)$$

Therefore, these equations can be generalized to L convolution layers and 1 pooling layer as follows:

$$Q_i = M_{i-1} - LC_{min} \sum_{m=i+1}^B S_0^{m-i} - (P_0 - L) \sum_{m=i}^B S_0^{m-i} - S_0^{(B-i+1)} \quad (41)$$

And we should have:

$$\sum_{k=1}^L C_{i,k} \leq Q_i \quad (42)$$

Similar to Eqs. (38), (39) and (42), intervals for $C_{i,1}$ and $C_{i,2}$ are computed as:

$$C_{i,1} \in [C_{min}, Q_i - (L - 1)C_{min}] \quad (43)$$

$$C_{i,2} \in [C_{min}, Q_i - C_{i,1} - (L - 2)C_{min}] \quad (44)$$

Finally, lower and upper bounds for k th convolution layer in i th block are concluded as:

$$C_{i,k} \in \left[C_{min}, Q_i - \sum_{j=1}^{k-1} C_{i,j} - (L - k)C_{min} \right] \quad (45)$$

It is worth mentioning that in the proposed CPOCNN model, lower and upper bounds of convolution kernel dimensions should be computed based on the order of layers as $C_{i,1}, C_{i,2}, \dots, C_{i,L}$. Steps of CPOCNN are described in Algorithm 1. Lines (1–4) check consistency of CNN structures with CPOCNN model. CNN particles with convolution, pooling and FC layers are randomly initialized

in lines (5–28). pBest and gBest are selected in lines (29–37). Parameters of particles are updated towards pBest and gBest (Lines 43–54). Particles are iteratively updated and gBest is selected in the last iteration (lines 35–62). Final gBest is trained with higher number of epochs several times and average accuracy is reported as final output of CPOCNN.

Algorithm 1 Proposed CPOCNN

Input: Training data (X), Test data (Y), Number of CNN blocks (B), Number of convolution layers in block i (NC_i), Number of pooling layers in block i (NP_i), Number of convolution layers for upper bound computation (L), Minimum dimension of convolution kernel (C_{min}), Maximum number of kernels (NK_{max}), Minimum number of kernels (NK_{min}), population size (N), Dimension of pooling kernel (P_0), Maximum number of neurons in FC layers ($nFCN_{max}$), Maximum number of FC layers ($nFCL_{max}$), Maximum number of iterations ($maxltr_{max}$), Weight of pBest in particles update (α_1), Weight of gBest in particles update (α_2), Number of epochs for CNN particles evaluation ($EpParCNN$), Number of epochs for optimized CNN training ($EpOpCNN$), Learning rate (lr), Batch size (BS), Optimization method (Opt), Number of optimized CNN training ($NOpCNNTr$).

Output: Average accuracy of optimized CNN.

Initialization of the CNN Particles:

```

1: if  $NC_i$  is not same for all blocks then
2:   Consider maximum of  $NC_i$  as  $L$ ;
3: if  $NP_i == g$  and  $g > 1$  for block  $i$  then
4:   Use same zero padding for  $m - 1$  pooling layers in block  $i$ 
;
5: for  $n = 1$  to  $N$  do
6:   for  $b = 1$  to  $B$  do
7:     for  $l = 1$  to  $L$  do
8:       Select random number  $RD$  in interval  $C_{b,l}$  by Eq.
(45);
9:       Select random number  $RN$  in interval  $[NK_{min} -$ 
 $NK_{max}]$ ;
10:       $Layers[b, l].Type \leftarrow Convolution$ ;
11:       $Layers[b, l].KerNum \leftarrow RN$ ;
12:       $Layers[b, l].KerDim \leftarrow RD$ ;
13:       $R \leftarrow rand(0, 1)$ ;
14:      if  $R < 0.5$  then
15:         $Layers[b, l + 1].Type \leftarrow MaxPooling$ ;
16:         $Layers[b, l + 1].KerDim \leftarrow P_0$ ;
17:      else
18:         $Layers[b, l + 1].Type \leftarrow AvePooling$ ;
19:         $Layers[b, l + 1].KerDim \leftarrow P_0$ ;

```

Numerical examples of the proposed model for four structures of CNNs with $L = 1$, $L = 2$, $L = 3$ and $L = 4$ are reported in Table 2 as Test 1–4. It shows that the proposed model keep data dimension as “non-empty” to be forwarded to the last layer. Also, flexible upper-bounds in CPOCNN reveals that parameters in the first layers are selected from bigger intervals and if they are selected as big values, intervals of the next layers are more limited. For example, in Test 1, 1343 is selected for $C_{1,1}$ from interval $[3, 1962]$ which shrinks interval of next layer to $[3, 313]$. Therefore, the highest degree of freedom is considered for parameter selection in each layer of CNN.

Property 1. CPOCNN explores more optimization space in comparison with all state-of-the art methods.

```

20:   $R_1 \leftarrow rand(0, 1)$ ;
21:   $NFCL = round(nFCL_{max} \times R_1)$ 
22:  for  $nFC = 1$  to  $NFCL$  do
23:     $R_2 \leftarrow rand(0, 1)$ ;
24:     $NFCN = round(nFCN_{max} \times R_2)$ ;
25:     $Layers[B + 1, nFC] \leftarrow FC$ ;
26:     $Layers[B + 1, nFC].NumNeuron \leftarrow NFCN$ ;
27:     $CNN_n.Layers \leftarrow Layers$ ;
28:  Save  $\{CNN_1, \dots, CNN_N\}$ 
CNN Structure Optimization:
29:   $num_{gBest} = 0$ ;
30:  for  $i = 1$  to  $N$  do
31:     $pBest_i \leftarrow CNN_i$ ;
32:     $Model \leftarrow Train(CNN_i, X, EpParCNN, optimizer, lr, BS)$ ;
33:     $CNN_i.Accuracy \leftarrow Evaluate(Model, Y)$ ;
34:   $gBest = CNN_1$ ;
35:  for  $i = 2$  to  $N$  do
36:    if  $gBest.Accuracy < CNN_i.Accuracy$  then
37:       $gBest = CNN_i$ ;
38:  for  $itr = 2$  to  $maxltr$  do
39:    for  $n = 1$  to  $N$  do
40:       $Layers = CNN_n.Layers$ ;
41:       $pBestLayers = pBest_i.Layers$ ;
42:       $gBestLayers = gBest.Layers$ ;
43:      for  $b = 1$  to  $B$  do
44:        for  $l = 1$  to  $L$  do
45:           $Layers[b, l].KerNum = Layers[b, l].KerNum +$ 
 $\alpha_1(pBestLayers[b, l].KerNum - Layers[b, l].KerNum) +$ 
 $\alpha_2(gBestLayers[b, l].KerNum - Layers[b, l].KerNum)$ ;
46:          if  $Layers[b, l].KerNum < NK_{min}$  then
47:             $Layers[b, l].KerNum \leftarrow NK_{min}$ ;
48:          if  $Layers[b, l].KerNum > NK_{max}$  then
49:             $Layers[b, l].KerNum \leftarrow NK_{max}$ ;


---


50:           $Layers[b, l].KerDim = Layers[b, l].KerDim +$ 
 $\alpha_1(pBestLayers[b, l].KerDim - Layers[b, l].KerDim) +$ 
 $\alpha_2(gBestLayers[b, l].KerDim - Layers[b, l].KerDim)$ ;
51:          if  $Layers[b, l].KerDim < C_{min}$  then
52:             $Layers[b, l].KerDim \leftarrow C_{min}$ ;
53:          if  $Layers[b, l].KerDim > Q_b - \sum_{i=1}^{l-1} C_{b,i} - (L-l)C_{min}$ 
then
54:             $Layers[b, l].KerDim \leftarrow Q_b - \sum_{i=1}^{l-1} C_{b,i} - (L-l)C_{min}$ ;
55:           $CNN_n.Layers \leftarrow Layers$ ;
56:           $Model \leftarrow Train(CNN_n, X, EpParCNN, Opt, lr, BS)$ ;
57:           $CNN_n.Accuracy \leftarrow Evaluate(Model, Y)$ ;
58:        for  $i = 1$  to  $N$  do
59:          if  $gBest.Accuracy < CNN_i.Accuracy$  then
60:             $gBest = CNN_i$ ;
61:          if  $pBest_i.Accuracy < CNN_i.Accuracy$  then
62:             $pBest_i = CNN_i$ ;
63:        for  $t = 1$  to  $NOpCNNTr$  do
64:           $Model \leftarrow Train(CNN_i, X, EpOpCNN, optimizer, lr, BS)$ ;
65:           $Acc[t].Accuracy \leftarrow Evaluate(Model, Y)$ ;
66:  Return average( $Acc$ );

```

Proof. All proposed CNN optimization methods in the literature use a constant interval for exploration of parameters in optimization space. The proposed CPOCNN calculates adaptive interval for each layer based on parameters of the previous layers and

Table 2Four examples of C_i determination in different layers of CNNs ($L = 1, 2, 3, 4$).

Test 1: $L = 1, B = 8, M_0 = 4000, C_{min} = 3, P_0 = 5, S_0 = 2$									
	Block	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$	$i = 8$
$k = 1$	$C_{min,1}$	3	3	3	3	3	3	3	3
	$C_{max,1}$	1962	313	63	29	7	5	4	4
	$C_{i,1}$	1343	195	13	23	5	5	3	3
	$M_{i,1}$	2658	1133	553	253	121	55	24	8
	M_i	1327	565	275	125	59	26	10	2
Test 2: $L = 2, B = 8, M_0 = 4000, C_{min} = 3, P_0 = 4, S_0 = 2$									
	Block	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$	$i = 8$
$k = 1$	$C_{min,1}$	3	3	3	3	3	3	3	3
	$C_{max,1}$	1707	319	22	9	4	4	4	4
	$C_{i,1}$	723	179	11	5	3	3	3	3
	$M_{i,1}$	3278	1282	577	282	135	63	27	9
$k = 2$	$C_{min,2}$	3	3	3	3	3	3	3	3
	$C_{max,2}$	987	143	14	7	4	4	4	4
	$C_{i,2}$	357	107	3	7	3	3	3	3
	$M_{i,2}$	2922	1176	575	276	133	61	25	7
	M_i	1460	587	286	137	65	29	11	2
Test 3: $L = 3, B = 8, M_0 = 4000, C_{min} = 3, P_0 = 2, S_0 = 2$									
	Block	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$	$i = 8$
$k = 1$	$C_{min,1}$	3	3	3	3	3	3	3	3
	$C_{max,1}$	1707	155	6	5	4	4	4	4
	$C_{i,1}$	151	107	3	5	3	3	3	3
	$M_{i,1}$	3850	1190	569	278	135	63	27	9
$k = 2$	$C_{min,2}$	3	3	3	3	3	3	3	3
	$C_{max,2}$	1559	51	6	3	4	4	4	4
	$C_{i,2}$	1251	41	3	3	3	3	3	3
	$M_{i,2}$	2600	1150	567	276	133	61	25	7
$k = 3$	$C_{min,3}$	3	3	3	3	3	3	3	3
	$C_{max,3}$	311	13	6	3	4	4	4	4
	$C_{i,3}$	9	9	3	3	3	3	3	3
	$M_{i,3}$	2592	1142	565	274	131	59	23	5
	M_i	1296	571	282	137	65	29	11	2
Test 4: $L = 4, B = 8, M_0 = 4000, C_{min} = 3, P_0 = 3, S_0 = 2$									
	Block	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$	$i = 8$
$k = 1$	$C_{min,1}$	3	3	3	3	3	3	3	3
	$C_{max,1}$	687	13	4	4	4	4	4	4
	$C_{i,1}$	485	5	3	3	3	3	3	3
	$M_{i,1}$	3516	1658	819	403	195	91	39	13
$k = 2$	$C_{min,2}$	3	3	3	3	3	3	3	3
	$C_{max,2}$	205	11	4	4	4	4	4	4
	$C_{i,2}$	57	9	3	3	3	3	3	3
	$M_{i,2}$	3460	1650	817	401	193	89	37	11
$k = 3$	$C_{min,3}$	3	3	3	3	3	3	3	3
	$C_{max,3}$	151	5	4	4	4	4	4	4
	$C_{i,3}$	103	5	3	3	3	3	3	3
	$M_{i,3}$	3358	1646	815	399	191	87	35	9
$k = 4$	$C_{min,4}$	3	3	3	3	3	3	3	3
	$C_{max,4}$	51	3	4	4	4	4	4	4
	$C_{i,4}$	33	3	3	3	3	3	3	3
	$M_{i,4}$	3326	1644	813	397	189	85	33	7
	M_i	1662	821	405	197	93	41	15	2

number of remained layers to reach output. In the other word, in previous methods, if a low value is selected for parameters of the first layers, it does not affect the parameters selection of subsequent layers while CPOCNN use bigger upper-bounds in this case. To show this property, consider a CNN with L convolution layers and input size M_0 . Based on Eq. (1), passing input image from the first layer with kernel size C_i creates output size of $M_0 - C_i + 1$. Previous methods use a constant upper-bound as large as input data is non-empty in all layers. Therefore, by considering $C_i = C_{max}$, when input data is passed from L layers, output data

Table 3

Parameters used in experiments.

C_{min}	NK _{max}	NK _{min}	BS	lr	Opt	N	nFCN _{max}
3	256	3	128	0.001	adam	20	$O \times 2$
nFCLmax	maxltr	α_1	α_2	EpParCNN	EpOpCNN	NOpCNNTr	
3	30	0.3	0.3	10	100	5	

Table 4Average and standard deviation of all measures in CPOCNN model with $B = 3$ and $L = 2$.

Dataset			CVX	MBI	MRB	MRD	MRDBI	RectI
CPOCNN ($L = 2$)	Prec	Ave	97.86	96.19	97.24	95.45	85.11	97.18
		Std	1.71	1.29	1.14	2.24	5.05	0.35
	SE	Ave	97.80	96.16	97.21	95.49	84.84	97.18
		Std	1.78	1.49	1.52	3.05	5.72	0.35
	SP	Ave	97.80	99.58	99.69	99.50	98.34	97.18
		Std	1.78	0.15	0.13	0.24	0.59	0.35
	ACC	Ave	97.80	96.17	97.92	95.44	86.91	97.18
		Std	0.04	1.14	0.98	2.24	4.80	0.00

should be non-empty as follows:

$$M_0 - L(C_{max,i} - 1) \geq 1 \quad (46)$$

It leads to the following condition for C_{max} :

$$C_{max,i} \leq \frac{M_0 + L - 1}{L} \quad (47)$$

Therefore, $C_{max,i}$ is upper-bound of C_i ($C_i \leq C_{max}$). C_{max} for all next layers j ($j = i + 1, i + 2, \dots, L$) is:

$$C_{max,(j)} = \left\lfloor \frac{M_0 + L - 1}{L} \right\rfloor \quad (48)$$

In CPOCNN, difference between C_i and C_{max} is taken into account and lower values of C_i leads to bigger C_{max} (upper-bound) of next layers as follows:

$$C_{max,CPOCNN,(i+1)} = \left\lfloor \frac{M_0 + L - 1}{L} \right\rfloor + (C_{max} - C_i) \quad (49)$$

Since $C_i \leq C_{max}$ and $C_{max} - C_i \geq 0$:

$$C_{max,CPOCNN,(i+1)} \geq C_{max,(i+1)} \quad (50)$$

Therefore, it can be concluded that CPOCNN considers bigger upper-bounds in comparison with other methods which leads to more exploration of optimization space. $\square$

3. Experimental setup and results

3.1. Datasets and parameters setting

Since the proposed model does not need any knowledge of domain, it can be applied in any data type. Therefore, 7 publicly available datasets: MNIST, MNIST Rotated Digits (MRD), MNIST Rotated Digits with Background Image (MRDBI), MNIST Random Background (MRB), MNIST Background Images (MBI), Convex (CVX) and Rectangles Images (RectI) have been selected for evaluation of CNN models. Parameters used in optimization algorithm are reported in Table 3.

3.2. Competitive methods and challenges of comparison

Three types of CNN models have been proposed in the literature. The first category is constant CNN structures, the second and third ones are CNNs with optimized structures and optimized

Table 5
Comparison between CPOCNN and competitive methods.

Model	CVX	MBI	MRB	MRD	MRDBI	RectI
MPSO [19]	90.33	–	–	–	65.77	–
CAE-1 [28]	–	83.30	86.43	88.41	51.90	78.14
CAE-2 [28]	–	84.50	89.10	90.34	54.77	78.46
PCANet-2 [29]	95.81	88.45	93.15	91.48	64.14	86.61
RandNet-2 [29]	94.55	88.35	86.53	91.53	56.31	83.00
LDANet-2 [29]	92.78	87.58	93.19	95.48	61.46	83.80
TIRBM [30]	–	–	–	93.80	64.50	–
PGBM [31]	–	87.75	93.92	–	63.24	–
ScatNet-2 [32]	93.50	81.60	87.70	92.52	49.52	–
SVMRBF [33]	80.87	77.39	85.42	89.62	67.38	–
SVMPoly [33]	80.18	75.99	83.38	86.39	62.41	–
NNet [33]	67.75	72.59	79.96	82.38	57.83	–
SAA-3 [33]	81.59	77.00	88.72	88.57	75.91	–
DBN-3 [33]	81.37	83.69	93.27	87.70	71.49	–
evoCNN [9]	94.61	95.38	96.41	84.54	62.62	94.03
IPPSO [13]	87.94	–	–	–	65.50	–
psoCNN [10]	96.10	97.48	97.47	83.58	79.02	96.06
BQ-CNN [34]	97.69	95.00	97.00	97.00	83.06	–
Proposed CPOCNN	97.80	96.17	97.92	95.44	86.91	97.18

Table 6
Comparison between CPOCNN and competitive methods in MNIST dataset.

MOPSO [11]	Deeply Supervised Net [35]	Shallow CNN [36]	Gated Pooling CNN [37]	IEA-CNN [18]	CEA-CNN, k = 1 [18]	CEA-CNN, k = 2 [18]
99.60	99.61	99.63	99.71	99.66	99.74	99.76
CEA-CNN, k = 3 [18]	FA-CNN [38]	TGA-CNN [39]	PSO [40]	FA [40]	CS [40]	BA [40]
99.72	99.13	99.14	99.17	99.16	99.14	99.13
EE-TGA-CNN [16]	E3FA-CNN [16]	PSO-mSDA [41]	1-layer DAE [41]	2-layer DAE [41]	mIDAE [42]	MPSO-CNN [19]
99.18	99.16	98.70	98.63	98.71	92.83	99.13
LeNet-1 [43]	LeNet-4 [43]	LeNet-5 [43]	RCNN [44]	SFCAE [45]	CAE-1 [28]	CAE-2 [28]
98.30	98.90	99.05	99.69	99.51	97.17	97.52
PCANet-2 [29]	RandNet-2 [29]	LDANet-2 [29]	ScatNet-2 [32]	SVM + RBF [33]	SVM + Poly [33]	NNet [33]
98.94	98.73	98.60	98.73	96.97	96.31	95.31
SAA-3 [33]	DBN-3 [33]	evoCNN(mean) [9]	IPPSO(mean) [13]	psoCNN(mean) [10]	BQ-CNN(mean) [34]	SSAE [46]
96.54	96.89	98.72	98.79	99.56	99.00	96.29
SDAE [47]	SCAE [48]	SCRBM [49]	CDAE [50]	OCHCNN		
99.06	99.29	99.18	99.38	99.77		

Table 7
Pair comparison between CNN models before and after optimization.

Dataset			CVX		MBI		MRB		MRD		MRDBI		RectI	
			B	A	B	A	B	A	B	A	B	A	B	A
DNACNN [51]	Prec	Ave	87.57	93.79	80.71	91.41	87.78	93.96	80.85	94.26	36.76	70.79	78.65	89.85
		Std	5.63	3.44	6.58	3.34	3.08	1.83	8.19	2.81	14.76	8.70	1.33	0.42
	SE	Ave	86.73	93.52	80.41	91.24	87.63	93.84	80.63	94.19	39.48	70.46	78.61	89.84
		Std	7.49	3.93	6.20	3.54	5.81	3.36	10.70	3.10	27.72	10.36	2.06	0.64
	SP	Ave	86.73	93.52	97.84	91.24	98.64	99.32	97.88	99.36	93.38	96.75	78.61	89.84
		Std	7.49	3.93	0.86	3.54	0.36	0.23	0.95	0.32	4.23	1.14	2.06	0.64
	ACC	Ave	86.65	93.51	80.43	91.28	87.62	93.87	80.53	94.19	35.38	70.40	78.59	89.84
		Std	1.00	0.25	5.56	2.72	3.81	2.01	8.65	2.24	21.78	8.78	0.37	0.11
LGG [52]	Prec	Ave	95.48	97.76	94.15	95.12	95.46	96.32	91.44	93.30	77.53	79.50	93.96	96.02
		Std	4.50	2.15	2.54	2.27	2.95	1.27	5.42	3.91	6.89	7.36	2.26	1.62
	SE	Ave	95.04	97.66	93.89	95.09	95.40	96.31	91.30	93.10	76.54	78.96	93.82	95.98
		Std	4.94	2.25	3.99	1.81	1.37	1.19	5.73	4.30	8.99	7.10	2.63	1.72
	SP	Ave	95.04	97.66	99.33	99.46	99.49	99.59	99.04	99.25	97.42	97.68	93.82	95.98
		Std	4.94	2.25	0.34	0.26	0.34	0.13	0.67	0.50	1.11	1.07	2.63	1.72
	ACC	Ave	95.02	97.66	93.93	95.09	95.40	96.31	91.26	93.13	76.65	79.01	93.82	95.97
		Std	0.25	0.05	1.77	1.45	1.68	1.04	4.73	3.37	5.77	5.87	0.19	0.05

(continued on next page)

Table 7 (continued).

Dataset			CVX		MBI		MRB		MRD		MRDBI		RectI	
			B	A	B	A	B	A	B	A	B	A	B	A
HGG [52]	Prec	Ave	95.85	97.15	94.45	96.03	95.20	96.40	91.94	93.77	80.78	82.16	95.04	96.33
		Std	4.13	2.55	2.84	1.87	2.87	1.76	5.82	3.91	9.16	6.38	1.81	0.46
	SE	Ave	95.48	97.01	94.31	95.86	95.08	96.38	91.66	93.76	79.90	81.59	94.98	96.33
		Std	4.51	2.71	3.28	2.15	3.86	1.80	4.34	3.07	7.58	6.70	1.96	0.53
	SP	Ave	95.48	97.01	99.37	99.55	99.46	99.60	99.08	99.31	97.78	97.97	94.98	96.33
		Std	4.51	2.71	0.35	0.24	0.36	0.20	0.71	0.45	1.30	0.98	1.96	0.53
	ACC	Ave	95.47	97.01	94.31	95.92	95.05	96.37	91.69	93.73	79.91	81.67	94.97	96.33
		Std	0.20	0.08	1.73	1.13	1.86	1.09	4.09	3.09	6.33	5.25	0.08	0.04
EEGCNN [53]	Prec	Ave	91.55	0.96	91.23	93.92	93.49	95.85	90.52	94.49	70.00	76.79	87.47	94.91
		Std	2.56	4.01	3.05	2.54	2.79	2.45	5.26	1.54	11.40	6.26	4.69	2.32
	SE	Ave	91.39	95.60	91.20	93.70	93.26	95.83	90.35	94.51	67.65	76.34	86.94	94.76
		Std	3.08	4.36	2.99	3.06	3.26	2.48	6.59	2.91	13.98	8.78	6.12	2.63
	SP	Ave	91.39	95.60	99.02	99.31	99.26	99.54	98.93	99.39	96.44	97.40	86.94	94.76
		Std	3.08	4.36	0.36	0.33	0.36	0.27	0.61	0.15	2.03	0.85	6.12	2.63
	ACC	Ave	91.39	95.59	91.15	93.76	93.30	95.79	90.21	94.49	67.36	76.29	86.87	94.77
		Std	0.27	0.19	1.88	1.79	1.49	1.30	4.21	2.04	9.32	6.25	0.75	0.16
OVACNN [54]	Prec	Ave	94.93	97.42	93.55	95.31	92.75	96.22	92.01	93.39	76.15	81.50	94.18	96.10
		Std	5.06	2.44	2.36	2.25	3.01	1.53	4.11	3.81	5.66	5.44	2.62	0.38
	SE	Ave	94.36	97.29	93.45	95.25	92.76	96.18	91.91	93.34	75.83	81.21	94.05	96.09
		Std	5.63	2.58	2.46	1.73	3.63	1.84	4.12	2.53	9.58	8.21	2.91	0.45
	SP	Ave	94.36	97.29	99.28	99.48	99.20	99.58	99.11	99.27	97.34	97.93	94.05	96.09
		Std	5.63	2.58	0.28	0.26	0.33	0.17	0.47	0.43	0.79	0.72	2.91	0.45
	ACC	Ave	94.34	97.29	93.48	95.26	92.71	96.19	91.89	93.35	75.69	81.16	94.03	96.09
		Std	0.32	0.07	1.99	1.56	2.67	1.25	3.29	2.99	6.32	5.80	0.15	0.03

Table 8

One epoch (20 iterations) evaluation of CNN particles.

ltr	CVX	MBI	MRB	MRD	MRDBI	RectI
1	41.47	86.39	87.76	79.18	89.30	46.36
2	39.74	85.34	88.38	77.13	89.11	44.86
3	39.05	85.28	87.85	76.61	89.23	44.36
4	37.86	84.34	88.37	76.45	89.22	43.38
5	36.10	82.13	87.20	76.40	89.11	42.31
6	38.02	83.48	88.25	76.09	88.71	43.65
7	36.85	83.51	87.59	77.03	88.59	46.14
8	36.02	83.32	87.25	77.10	89.09	45.30
9	38.26	82.66	88.10	75.38	89.23	44.27
10	37.30	81.96	87.79	77.29	88.59	43.41
11	38.68	83.39	87.43	75.02	89.58	45.39
12	37.39	83.66	86.27	76.43	89.73	43.68
13	36.99	83.59	86.26	76.21	89.38	43.66
14	38.08	82.45	87.80	76.52	89.81	43.94
15	38.22	83.94	86.76	77.19	89.48	46.40
16	39.07	83.97	87.26	75.70	89.26	45.75
17	40.09	84.15	87.31	77.08	89.39	42.75
18	38.63	83.13	86.97	76.64	89.45	44.67
19	37.07	81.60	86.64	76.70	89.34	44.26
20	37.44	82.25	86.38	75.13	89.32	45.61

parameters, respectively. These categories are completely different and their comparison may be unfair. To compare CNNs of all categories, performance of final structure in each model is considered as a measure of evaluation. Note that performance of CNNs has not been reported in some datasets, therefore, these cases have been excluded from evaluation. Based on mentioned limitations and challenges for comparison, two types of evaluation are designed in this research as follows.

In the first one, CNNs are constructed based on the proposed CPOCNN models and then optimized. Optimized CPOCNN models are compared with competitive methods including: MPPO [19], CAE-1 [28], CAE-2 [28], PCANet-2 [29], RandNet-2 [29], LDANet-2 [29], TIRBM [30], PGBM [31], ScatNet-2 [32], SVMRF [33], SVMPoly [33], NNet [33], SAA-3 [33], DBN-3 [33], evoCNN [9], IPPSO [13], psoCNN [10], BQ-CNN [34], MOPSO [11], Deeply Supervised Net [35], Shallow CNN [36], Gated Pooling CNN [37], IEA-CNN [18], CEA-CNN, $k = 1$ [18], CEA-CNN, $k = 2$ [18,38],

Table 9

10 epochs evaluation of CNN particles.

Epoch	CVX	MBI	MRB	MRD	MRDBI	RectI
1	38.09	83.99	83.82	75.87	86.72	48.87
2	18.84	50.95	43.79	54.53	74.2	32.64
3	14.61	29.12	20.21	34.97	63.92	24.31
4	12.67	19.36	13.87	18.62	54.56	19.8
5	9.93	13.25	10.91	12.59	47.34	15.47
6	8.21	11.48	9.32	9.45	41.47	12.44
7	8.2	8.85	7.88	6.95	34.28	10.3
8	7.48	7.82	6.79	5.93	29.26	8.28
9	5.06	6.3	5.94	5.02	25.78	7.14
10	4.51	5.69	5.44	3.92	21.43	5.94

Table 10

Structure of base models.

Model	1	Conv	Conv	Conv	Conv	Conv	Conv	Conv	Conv	Conv	FC
Model 2	Conv	Conv	Conv	Pool	Conv	Conv	Conv	Pool	FC	–	–
Model 3	Conv	Conv	Pool	Conv	Conv	Pool	Conv	Conv	Pool	FC	–
Model 4	Conv	Pool	Conv	Pool	Conv	Pool	Conv	Pool	FC	–	–

TGA-CNN [39], PSO [40], FA [40], CS [40], BA [40], EE-TGA-CNN [16], E3FA-CNN [16], PSO-mSDA [41], 1-layer DAE [41], 2-layer DAE [41], mDAE [42], MPPO-CNN [19], LeNet-1 [43], LeNet-4 [43], LeNet-5 [43], RCNN [44], SFCAE [45], SSAE [46], SDAE [47], SCAE [48], SCRBM [49] and CDAE [50].

In the second comparison scheme, CNN models DNACNN [51], LGG [52], HGG [52], EEGCNN [53] and OVACNN [54] have been selected and then optimized by CPOCNN. Then, a pairwise comparison is done between primary CNNs and optimized CNNs by CPOCNN.

3.3. Type 1: Comparison with competitive CNNs

CNNs with sequential blocks of L convolution layers and 1 pooling layer can be optimized by the proposed CPOCNN models described in Section 2. Here one case with $L = 2$ is considered for evaluation. Other cases will be investigated in Discussion Section.

Table 11
Average and standard deviation of all measures in CPOCNN models.

Dataset			CVX	MBI	MRB	MRD	MRDBI	RectI
CPOCNN1	Prec	Ave	96.31	94.14	95.89	91.08	76.53	97.02
		Std	2.89	2.71	1.77	4.21	7.68	0.14
	SE	Ave	96.13	94.05	95.88	91.07	76.25	97.02
		Std	3.12	2.68	1.30	4.96	8.93	0.12
	SP	Ave	96.13	99.34	99.55	99.02	97.38	97.02
		Std	3.12	0.30	0.19	0.46	1.02	0.12
	ACC	Ave	96.13	94.05	95.88	91.04	76.19	97.02
		Std	0.12	1.75	1.47	4.31	7.54	0.01
CPOCNN2	Prec	Ave	97.49	96.79	97.55	94.97	84.89	97.21
		Std	2.46	0.96	1.13	2.85	4.41	0.26
	SE	Ave	97.36	96.78	97.57	94.89	84.78	97.21
		Std	2.59	1.28	0.91	3.09	6.93	0.25
	SP	Ave	97.36	99.64	99.73	99.44	98.32	97.21
		Std	2.59	0.10	0.12	0.33	0.53	0.25
	ACC	Ave	97.36	96.78	97.56	94.91	84.71	97.21
		Std	0.07	0.98	0.81	2.64	4.98	0.01
CPOCNN4	Prec	Ave	95.35	93.85	95.95	95.30	79.00	93.68
		Std	1.41	2.42	1.63	2.46	6.69	0.16
	SE	Ave	95.31	93.79	95.92	95.20	78.87	93.67
		Std	1.56	2.54	2.36	2.26	7.16	0.24
	SP	Ave	95.31	99.31	99.55	99.47	97.67	93.67
		Std	1.56	0.29	0.19	0.30	0.81	0.24
	ACC	Ave	95.31	93.79	95.91	95.23	78.83	93.67
		Std	0.07	1.98	1.43	2.00	6.45	0.04

In the CPOCNN model with $L = 2$, each block has two convolution layers and one pooling layer. In this experiment three blocks are considered ($B = 3$). Parameters of models are optimized by CPOCNN and reported in Table 12.

CPOCNN has been applied to these models and leads to optimized models CPOCNN($L = 2$). It optimizes convolution parameters including number and dimension of convolution kernels. Remained parameters of CNNs are not changed during optimization process except type of pooling which is selected randomly as Ave/Max pooling. Average and standard deviation of Precision, Sensitivity, Specificity and Accuracy criteria of CPOCNN on each dataset are reported in Table 4. Since in competitive methods, only accuracy is reported, accuracy of CPOCNN has been compared with all methods in Table 5. Among 18 constant and optimized CNN models, CPOCNN achieves the first rank by average accuracies 97.80%, 97.92%, 86.91% and 97.18% in Convex, MNIST-BI, MNIST-RB, MNIST-RDBI and Rectangle-Image datasets, respectively. In MNIST-BI and MNIST-RD, psoCNN and BQ-CNN are the best methods, respectively. Since results of many methods were available on MNIST dataset, comparison in this dataset is done in the separate Table 6 to save the space of Table 5. CPOCNN with accuracy of 99.77%, outperforms 46 competitive CNN models.

3.4. Type 2: Pairwise comparison

Second type of CPOCNN evaluation is pairwise comparison. For this task, 5 CNN models DNACNN, LGG, HGG, EEGCNN and OVACNN are optimized by CPOCNN and the performance of these CNNs in all measures before (B) and after (A) optimization are reported in Table 7. Optimizing CNNs with CPOCNN improves their performance in all measures. Interested conclusion of this evaluation is that average accuracy of some models such as DNACNN has been increased from 35.38% before optimization to 70.40% after optimization which reveals the significant effect of parameters in the performance of CNN models.

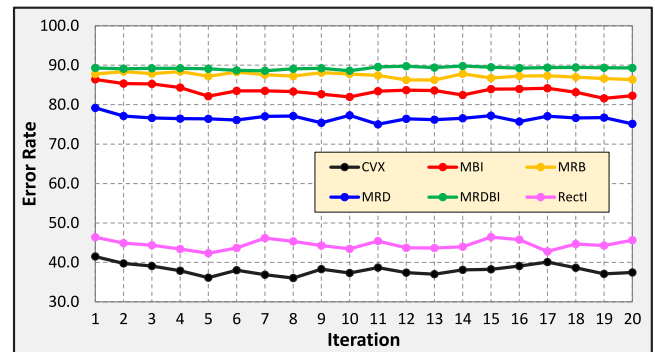


Fig. 3. Error rate trends in one epoch CNN evaluation.

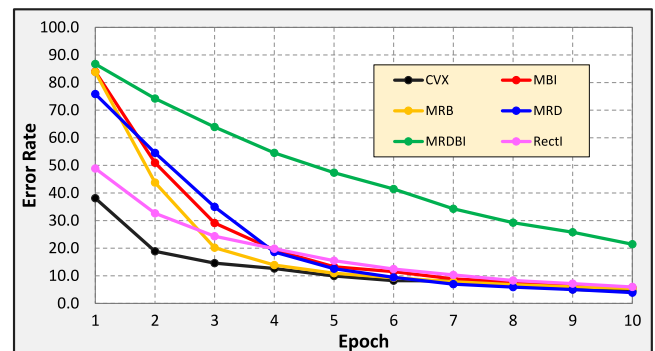


Fig. 4. Error rate trends in 10 epochs CNN evaluation.

4. Discussion

4.1. CNN particles evaluation

In all CNN optimization methods based on evolutionary algorithms, particle evaluation is a challenging part. Consider an

Table 12
Parameters of models optimized by CPOCNN in each dataset.

Dataset		CVX	MBI	MRB	MRD	MRDBI	RectI
		Param.	Param.	Param.	Param.	Param.	Param.
Model 1	Conv	(4,90)	(4, 100)	(6, 172)	(4, 179)	(4, 138)	(4, 72)
	Conv	(5, 134)	(3, 192)	(4, 111)	(5, 159)	(3, 134)	(3, 194)
	Conv	(5, 138)	(3, 92)	(5, 95)	(4, 126)	(4, 117)	(4, 144)
	Conv	(3, 151)	(3, 162)	(5, 132)	(4, 125)	(6, 137)	(4, 130)
	Conv	(4, 112)	(6, 137)	(5, 113)	(4, 110)	(4, 109)	(5, 96)
	Conv	(4, 129)	(3, 141)	(6, 146)	(4, 142)	(4, 131)	(4, 75)
	Conv	(5, 126)	(6, 84)	(4, 142)	(6, 198)	(6, 132)	(4, 130)
	Conv	(6, 81)	(6, 94)	(6, 130)	(4, 165)	(6, 137)	(3, 107)
	Conv	(5, 98)	(5, 160)	(4, 36)	(4, 164)	(5, 176)	(4, 90)
	FC	2	10	10	10	10	2
Model 2	Conv	(6, 121)	(4, 39)	(4, 103)	(3, 74)	(3, 160)	(3, 44)
	Conv	(4, 136)	(5, 142)	(4, 178)	(3, 105)	(3, 190)	(3, 160)
	Conv	(6, 151)	(4, 104)	(4, 168)	(4, 133)	(4, 193)	(3, 148)
	Pool	MP(2)	MP(2)	MP(2)	MP(2)	MP(2)	AP(2)
	Conv	(4, 86)	(3, 177)	(4, 146)	(4, 77)	(3, 131)	(4, 96)
	Conv	(5, 121)	(3, 122)	(5, 154)	(6, 123)	(6, 72)	(4, 79)
	Conv	(5, 156)	(5, 134)	(4, 174)	(5, 150)	(6, 126)	(6, 217)
	Pool	MP(2)	MP(2)	AP(2)	AP(2)	AP(2)	AP(2)
	FC	2	10	10	10	10	2
Model 3	Conv	(6,30)	(3,59)	(3,136)	(5,112)	(6,55)	(4,78)
	Conv	(5,176)	(4,93)	(3,163)	(4,137)	(5,112)	(3,112)
	Pool	MP(3)	AP(3)	AP(3)	AP(3)	MP(3)	MP(3)
	Conv	(4,102)	(3,154)	(4,164)	(3,158)	(3,135)	(3,197)
	Conv	(4,43)	(4,131)	(4,176)	(4,165)	(5,170)	(4,176)
	Pool	AP(3)	MP(3)	AP(3)	MP(3)	AP(3)	MP(3)
	Conv	(5,96)	(4,174)	(4,110)	(5,166)	(5,146)	(4,228)
	Conv	(6,63)	(4,184)	(4,167)	(4,195)	(4,231)	(4,179)
	Pool	AP(3)	MP(3)	MP(3)	MP(3)	MP(3)	MP(3)
	FC	2	10	10	10	10	2
Model 4	Conv	(4, 35)	(4, 70)	(5, 70)	(4, 116)	(3, 69)	(5, 176)
	Pool	MP(2)	MP(2)	AP(2)	MP(2)	AP(2)	MP(2)
	Conv	(5, 199)	(4, 158)	(4, 180)	(4, 148)	(6, 148)	(3, 162)
	Pool	AP(2)	MP(2)	MP(2)	MP(2)	MP(2)	MP(2)
	Conv	(4, 138)	(3, 108)	(5, 114)	(5, 127)	(4, 140)	(3, 180)
	Pool	MP(2)	MP(2)	AP(2)	AP(2)	AP(2)	AP(2)
	Conv	(4, 119)	(5, 151)	(6, 158)	(6, 139)	(4, 234)	(5, 80)
	Pool	MP(2)	MP(2)	MP(2)	MP(2)	MP(2)	MP(2)
	FC	2	10	10	10	10	2

Table 13
Run times of CPOCNN models (in seconds).

Model/Dataset	CVX	MBI	MRB	MRD	MRDBI	RectI
CPOCNN1	3042.29	4208.51	4175.15	3995.41	4243.70	3721.56
CPOCNN2	4356.88	6832.78	6734.96	6914.17	7236.51	4424.23
CPOCNN3	2392.33	3242.42	3841.67	3780.41	3988.68	2776.30
CPOCNN4	2862.51	3911.89	4199.42	5013.51	4222.23	3023.63

CNN optimization model based on PSO with N CNN particles, I iterations, E epochs and M iterations per epoch for each CNN particle evaluation. The whole optimization process needs $(N \times I \times E \times M)$ number of CNN training and evaluation which is a very time-consuming task. Therefore, to reduce this cost, CNNs are evaluated with low number of epochs and final optimized CNN is trained with higher epochs. For example, in [10], 1 epoch was used for CNN evaluation in optimization process and 100 epochs for training of final optimized CNN. In this research, error rate of 1 epoch (20 iterations) evaluation in all datasets are reported in Table 8. Also, trends of these errors are depicted in Fig. 3. These experiments reveal that 1 epoch does not force CNN particles to decrease test error rate in subsequent iterations while 10 epochs evaluation creates a downward trend in error rate of CNN particles (see Table 9 and Fig. 4). Therefore, in CPOCNN, 10-epochs training was used for CNN particles evaluation.

4.2. Models with different values of L

Different values of L means different number of convolution layers in each CNN block. To show the effectiveness of different structures in CPOCNN, a complete distribution of convolution layers alongside CNN structures are considered as base models 1–4: (1) all convolution, (2) $\frac{3}{4}$ convolution $\frac{1}{4}$ pooling, (3) $\frac{2}{3}$ convolution $\frac{1}{3}$ pooling and (4) $\frac{1}{2}$ convolution $\frac{1}{2}$ pooling, reported in Table 10. Note that Model 3 was used in the previous section. All measures of Models 1, 2 and 4 have been reported in Table 11 which show high accuracy, precision, sensitivity and specificity of optimized models in all datasets.

4.3. Optimized/freeze architectures

Optimization of CNN models for all datasets needs high computational cost. This challenge was addressed in Section 4.1 by using low number of epochs in particle evaluation phase. Other optimization models use the same strategy. It can be seen that training data of each dataset is the main input in Algorithm 1. It means that the proposed CPOCNN model and other optimization models find optimized architecture for each dataset independently and they do not use the same (freeze) optimized architecture for all datasets. CNNs optimized by the proposed CPOCNN in each dataset have been reported in Table 12.

Table 14
AlexNet optimized by CPOCNN.

Dataset		CVX		MBI		MRB		MRD		RectI	
		ANet	CPOCNN	ANet	CPOCNN	ANet	CPOCNN	ANet	CPOCNN	ANet	CPOCNN
Prec	Ave	89.05	91.61	79.03	79.83	81.72	82.93	86.92	87.75	81.54	82.77
	Std	5.12	5.67	12.90	8.25	10.62	8.66	6.05	5.05	0.18	2.54
SE	Ave	88.38	90.84	76.45	78.94	80.97	82.21	86.85	87.73	81.54	82.61
	Std	6.56	6.81	10.06	9.31	5.46	6.47	6.32	5.32	0.50	3.68
SP	Ave	88.38	90.84	97.40	97.68	97.89	98.04	98.55	98.65	81.54	82.61
	Std	6.56	6.81	0.02	1.28	1.38	1.21	0.70	0.57	0.50	3.68
ACC	Ave	88.33	90.80	76.51	78.92	81.06	82.30	86.77	87.71	81.54	82.57
	Std	0.77	0.63	8.01	6.71	7.12	6.27	5.39	4.94	0.16	0.58

4.4. Run time

As discussed before, different number of convolution layers in each CNN block makes CPOCNN models. Run time of CPOCNN models are reported in Table 13. Generally, CPOCNN3 with $\frac{2}{3}$ convolution and $\frac{1}{3}$ pooling layers in each block has the lowest run time. Also, $\frac{3}{4}$ convolution and $\frac{1}{4}$ pooling in CPOCNN2 needs the highest convergence time in optimization process. Finally, CPOCNN3 in Convex dataset and CPOCNN2 in MRDBI with run times 2392.33s and 7236.51s are the fastest and slowest models, respectively.

4.5. Limitations and assumptions of the proposed CPOCNN

CPOCNN has some limitations and assumptions in optimization process. The first limitation is that CPOCNN can only optimize CNNs with the constant block structures. It means that if the first block contains 2 convolution layers and 1 pooling layer, this structure should be repeated in subsequent blocks. Another limitation is that CPOCNN optimize just parameters in convolution layer and parameters in other layers are considered as constant values in optimization process. The last limitation is that beside parameter optimization, network architecture is not optimized. Therefore, order type and number of layers are constant during optimization process. These limitations are directed as future works in the next section.

4.6. Future works

4.6.1. Optimizing CNNs with different block structures

Proposed CPOCNN model uses same number of convolution (1, 2, ..., L) layers in each block. For CNN structures with different convolution layers in each block, maximum number of convolution layers in blocks are considered for construction of the model (Lines 1–2 in Algorithm 1 check this condition). AlexNet is an example of such cases which has one convolution layer in the first and second blocks and 3 convolution layers in the last block. Table 14 reports all measures of AlexNet before and after optimization in 5 datasets Convex, MNIST, MNIST-BI, MNIST-RB, MNIST-RD and Rectangle-Image. The best improvement of 2.47% in Convex dataset is achieved after optimization which is lower than improvement of CNNs with same block structures. Therefore, proposing an optimization model for CNNs with different block structures is directed as a future work which can improve CNN performance significantly.

4.6.2. Optimization of all parameters

The main contribution of CPOCNN is to adaptive determination of convolution kernel dimension in CNNs. CNN architecture has other parameters such as pooling kernel dimension, stride and zero-padding size which affect data dimension. These parameters are considered constant in CPOCNN. To achieve better results and

a comprehensive model, considering all parameters of CNN in optimization process is proposed as another future work. The primary model for output dimension computation of this suggestion can be proposed as:

$$M_i = \left\lfloor \frac{M_{i-1} + Z_i - C_i + 1 - P_i + S_i}{S_i} \right\rfloor \quad (51)$$

where M_i , M_{i-1} , Z_i , C_i , P_i and S_i are dimension of output image, dimension of input image, zero padding value, dimension of convolution kernel, dimension of pooling and stride value for i th block of CNN, respectively.

5. Conclusion

This paper presented a convolutional neural network (CNNs) parameters optimization method called CPOCNN. For this purpose, a mathematical model for upper-bounds of convolution filter dimension in each CNN block was proposed. Intervals of this parameter in each layer are computed based on data dimension of that layer and number of layers remained to reach output layer. It was proven that CPOCNN explores more optimization space than other optimization models. Results of image classification in several datasets showed that the proposed method outperforms existing state-of-the-art CNN models with constant and optimized structures. Since CPOCNN was developed for CNNs with same blocks, future efforts can be directed towards optimization of CNNs with different block structure. Future efforts will also be directed towards optimization of other CNN parameters.

CRedit authorship contribution statement

Milad Kohzadi Chegeni: Methodology, Software, Validation, Data curation. **Abdolreza Rashno:** Conceptualization, Validation, Writing – original draft, Supervision. **Sadegh Fadaei:** Conceptualization, Formal analysis, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai, et al., Recent advances in convolutional neural networks, *Pattern Recognit.* 77 (2018) 354–377.
- [2] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Adv. Neural Inf. Process. Syst.* 25 (2012).

- [3] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, arXiv preprint arXiv:1409.1556.
- [4] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [5] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.
- [6] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 4700–4708.
- [7] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean, Efficient neural architecture search via parameters sharing, in: *International Conference on Machine Learning*, PMLR, 2018, pp. 4095–4104.
- [8] Y. Li, J. Xiao, Y. Chen, L. Jiao, Evolving deep convolutional neural networks by quantum behaved particle swarm optimization with binary encoding for image classification, *Neurocomputing* 362 (2019) 156–165.
- [9] Y. Sun, B. Xue, M. Zhang, G.G. Yen, Evolving deep convolutional neural networks for image classification, *IEEE Trans. Evol. Comput.* 24 (2) (2019) 394–407.
- [10] F.E.F. Junior, G.G. Yen, Particle swarm optimization of deep neural networks architectures for image classification, *Swarm Evol. Comput.* 49 (2019) 62–74.
- [11] J. Jiang, F. Han, Q. Ling, J. Wang, T. Li, H. Han, Efficient network architecture search via multiobjective particle swarm optimization based on decomposition, *Neural Netw.* 123 (2020) 305–316.
- [12] B. Baker, O. Gupta, N. Naik, R. Raskar, Designing neural network architectures using reinforcement learning, 2016, arXiv preprint arXiv:1611.02167.
- [13] B. Wang, Y. Sun, B. Xue, M. Zhang, Evolving deep convolutional neural networks by variable-length particle swarm optimization for image classification, in: *2018 IEEE Congress on Evolutionary Computation (CEC)*, IEEE, 2018, pp. 1–8.
- [14] L. Xie, A. Yuille, Genetic cnn, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 1379–1388.
- [15] Y. Wang, H. Zhang, G. Zhang, cPSO-CNN: An efficient PSO-based algorithm for fine-tuning hyper-parameters of convolutional neural networks, *Swarm Evol. Comput.* 49 (2019) 114–123.
- [16] N. Bacanin, T. Bezdan, E. Tuba, I. Strumberger, M. Tuba, Optimizing convolutional neural network hyperparameters by enhanced swarm intelligence metaheuristics, *Algorithms* 13 (3) (2020) 67.
- [17] T. Yamasaki, T. Honma, K. Aizawa, Efficient optimization of convolutional neural networks using particle swarm optimization, in: *2017 IEEE Third International Conference on Multimedia Big Data (BigMM)*, IEEE, 2017, pp. 70–73.
- [18] E. Bochinski, T. Senst, T. Sikora, Hyper-parameter optimization for convolutional neural network committees based on evolutionary algorithms, in: *2017 IEEE International Conference on Image Processing (ICIP)*, IEEE, 2017, pp. 3924–3928.
- [19] P. Singh, S. Chaudhury, B.K. Panigrahi, Hybrid MPSONN: Multi-level particle swarm optimized hyperparameters of convolutional neural network, *Swarm Evol. Comput.* 63 (2021) 100863.
- [20] H. Louati, S. Bechikh, A. Louati, C.-C. Hung, L.B. Said, Deep convolutional neural network architecture design as a bi-level optimization problem, *Neurocomputing* 439 (2021) 44–62.
- [21] N.M. Aszemi, P. Dominic, Hyperparameter optimization in convolutional neural network using genetic algorithms, *Int. J. Adv. Comput. Sci. Appl.* 10 (6) (2019).
- [22] M. Karthiga, V. Santhi, S. Sountharajan, Hybrid optimized convolutional neural network for efficient classification of ECG signals in healthcare monitoring, *Biomed. Signal Process. Control* 76 (2022) 103731.
- [23] P. Muthukannan, et al., Optimized convolution neural network based multiple eye disease detection, *Comput. Biol. Med.* (2022) 105648.
- [24] M. Ramirez-Gonzalez, F.R.S. Sevilla, P. Korba, R. Castellanos-Bustamante, Convolutional neural nets with hyperparameter optimization and feature importance for power system static security assessment, *Electr. Power Syst. Res.* 211 (2022) 108203.
- [25] G. Habib, S. Qureshi, Optimization and acceleration of convolutional neural networks: A survey, *J. King Saud Univ.-Comput. Inf. Sci.* (2020).
- [26] H. Louati, A. Louati, S. Bechikh, F. Masmoudi, A. Aldaei, E. Kariri, Topology optimization search of deep convolution neural networks for CT and X-ray image classification, *BMC Med. Imaging* 22 (1) (2022) 1–11.
- [27] N.M.H. Alamri, M. Packianather, S. Bigot, Deep learning: parameter optimization using proposed novel hybrid bees Bayesian convolutional neural network, *Appl. Artif. Intell.* (2022) 1–26.
- [28] R. Salah, P. Vincent, X. Muller, et al., Contractive auto-encoders: Explicit invariance during feature extraction, in: *Proc. of the 28th International Conference on Machine Learning*, 2011, pp. 833–840.
- [29] T.-H. Chan, K. Jia, S. Gao, J. Lu, Z. Zeng, Y. Ma, PCANet: A simple deep learning baseline for image classification? *IEEE Trans. Image Process.* 24 (12) (2015) 5017–5032.
- [30] K. Sohn, H. Lee, Learning invariant representations with local transformations, 2012, arXiv preprint arXiv:1206.6418.
- [31] K. Sohn, G. Zhou, C. Lee, H. Lee, Learning and selecting features jointly with point-wise gated Boltzmann machines, in: *International Conference on Machine Learning*, PMLR, 2013, pp. 217–225.
- [32] J. Bruna, S. Mallat, Invariant scattering convolution networks, *IEEE Trans. Pattern Anal. Mach. Intell.* 35 (8) (2013) 1872–1886.
- [33] H. Larochelle, D. Erhan, A. Courville, J. Bergstra, Y. Bengio, An empirical evaluation of deep architectures on problems with many factors of variation, in: *Proceedings of the 24th International Conference on Machine Learning*, 2007, pp. 473–480.
- [34] Y. Li, J. Xiao, Y. Chen, L. Jiao, Evolving deep convolutional neural networks by quantum behaved particle swarm optimization with binary encoding for image classification, *Neurocomputing* 362 (2019) 156–165.
- [35] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, Z. Tu, Deeply-supervised nets, in: *Artificial Intelligence and Statistics*, PMLR, 2015, pp. 562–570.
- [36] M.D. McDonnell, T. Vladusich, Enhanced image classification with a fast-learning shallow convolutional neural network, in: *2015 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2015, pp. 1–7.
- [37] C.-Y. Lee, P.W. Gallagher, Z. Tu, Generalizing pooling functions in convolutional neural networks: Mixed, gated, and tree, in: *Artificial Intelligence and Statistics*, PMLR, 2016, pp. 464–472.
- [38] I. Strumberger, E. Tuba, N. Bacanin, M. Zivkovic, M. Beko, M. Tuba, Designing convolutional neural network architecture by the firefly algorithm, in: *2019 International Young Engineers Forum (YEF-ECE)*, IEEE, 2019, pp. 59–65.
- [39] I. Strumberger, E. Tuba, N. Bacanin, R. Jovanovic, M. Tuba, Convolutional neural network architecture design by the tree growth algorithm framework, in: *2019 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2019, pp. 1–8.
- [40] G.H. De Rosa, J.P. Papa, X.-S. Yang, Handling dropout probability estimation in convolution neural networks using meta-heuristics, *Soft Comput.* 22 (18) (2018) 6147–6156.
- [41] C. Sui, M. Bennamoun, R. Togneri, Deep feature learning for dummies: A simple auto-encoder training method using Particle Swarm Optimisation, *Pattern Recognit. Lett.* 94 (2017) 75–80.
- [42] J. Park, I.W. Sandberg, Universal approximation using radial-basis-function networks, *Neural Comput.* 3 (2) (1991) 246–257.
- [43] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324.
- [44] M. Liang, X. Hu, Recurrent convolutional neural network for object recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3367–3375.
- [45] Y. Sun, B. Xue, M. Zhang, G.G. Yen, A particle swarm optimization-based flexible convolutional autoencoder for image classification, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (8) (2018) 2295–2309.
- [46] H.-C. Shin, M.R. Orton, D.J. Collins, S.J. Doran, M.O. Leach, Stacked autoencoders for unsupervised feature learning and multiple organ detection in a pilot study using 4D patient data, *IEEE Trans. Pattern Anal. Mach. Intell.* 35 (8) (2012) 1930–1943.
- [47] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.-A. Manzagol, L. Bottou, Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion, *J. Mach. Learn. Res.* 11 (12) (2010).
- [48] J. Masci, U. Meier, D. Cireşan, J. Schmidhuber, Stacked convolutional autoencoders for hierarchical feature extraction, in: *International Conference on Artificial Neural Networks*, Springer, 2011, pp. 52–59.
- [49] H. Lee, R. Grosse, R. Ranganath, A.Y. Ng, Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations, in: *Proceedings of the 26th Annual International Conference on Machine Learning*, 2009, pp. 609–616.
- [50] B. Du, W. Xiong, J. Wu, L. Zhang, L. Zhang, D. Tao, Stacked convolutional denoising auto-encoders for feature representation, *IEEE Trans. Cybern.* 47 (4) (2016) 1017–1027.
- [51] Ü. Atila, Y.Y. Baydilli, E. Sehirli, M.K. Turan, Classification of DNA damages on segmented comet assay images using convolutional neural network, *Comput. Methods Programs Biomed.* 186 (2020) 105192.
- [52] S. Pereira, A. Pinto, V. Alves, C.A. Silva, Brain tumor segmentation using convolutional neural networks in MRI images, *IEEE Trans. Med. Imaging* 35 (5) (2016) 1240–1251.
- [53] S.-E. Moon, C.-J. Chen, C.-J. Hsieh, J.-L. Wang, J.-S. Lee, Emotional EEG classification using connectivity features and convolutional neural networks, *Neural Netw.* 132 (2020) 96–107.
- [54] Ö. İnik, A. Ceyhan, E. Balcioglu, E. Ülker, A new method for automatic counting of ovarian follicles on whole slide histological images based on convolutional neural network, *Comput. Biol. Med.* 112 (2019) 103350.